ADOPT-02: Platform Health Assessment

> **Series:** ADOPT — Observability Adoption & Maturity | **Notebook:** 2 of 6 | **Created:** March 2026 | **Last Updated:** 07/08/2026

Overview

A healthy observability platform is the foundation for everything else — alerting, troubleshooting, capacity planning, and automation all depend on complete and accurate data. This notebook provides a structured approach to assessing Dynatrace platform health: OneAgent deployment coverage, data ingestion rates, entity monitoring completeness, ActiveGate status, and license consumption. The result is a platform health scorecard you can review regularly.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	`storage:entities:read`, `storage:metrics:read`, `storage:logs:read`, `storage:events:read`
Data	At least 24 hours of OneAgent data
Audience	Platform engineers, Dynatrace administrators

1. OneAgent Deployment Coverage

OneAgent coverage is the most fundamental health indicator. Gaps in deployment mean gaps in visibility. The goal is to understand how many hosts are monitored and whether any are running outdated agent versions.

1.1 Total Monitored Hosts

```
// Count all monitored hosts and break down by monitoring mode
fetch dt.entity.host
| summarize total_hosts = count()
// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes HOST
// | summarize total_hosts = count()
```

1.2 Hosts by Monitoring Mode

Dynatrace supports multiple monitoring modes. Full-stack provides the deepest visibility, while infrastructure-only and cloud-only modes have limited capabilities.

```
// Break down hosts by monitoring mode
fetch dt.entity.host
| summarize host_count = count(), by:{monitoringMode}
| sort host_count desc
```

1.3 Agent Version Distribution

Running outdated OneAgent versions can lead to missed features, security vulnerabilities, and compatibility issues. This query identifies the distribution of agent versions across your environment.

```
// Identify OneAgent version distribution across hosts
fetch dt.entity.host
| summarize host_count = count(), by:{agentVersion}
| sort host_count desc
| limit 20
```

> **Tip:** If you see more than 3-4 distinct agent versions, consider enabling auto-update or scheduling a coordinated update. A single major version across the fleet reduces support complexity.

2. Host Monitoring Health

Beyond simple counts, we need to verify that monitored hosts are actively reporting data. A host entity may exist but stop sending metrics if the agent crashes or the host is decommissioned.

2.1 Host CPU Utilization Overview

This query provides a quick health check — if hosts are reporting CPU metrics, the agent is functional.

```
// Top 10 hosts by average CPU usage over last 1 hour
timeseries avgCpu = avg(dt.host.cpu.usage), from:-1h, by:{dt.entity.host}
| fieldsAdd avgCpuValue = arrayAvg(avgCpu)
| sort avgCpuValue desc
| limit 10
```

2.2 Host Memory Utilization

```
// Top 10 hosts by average memory usage over last 1 hour
timeseries avgMem = avg(dt.host.memory.usage), from:-1h, by:{dt.entity.host}
| fieldsAdd avgMemValue = arrayAvg(avgMem)
| sort avgMemValue desc
| limit 10
```

3. Service and Process Discovery

Auto-discovered services and process groups indicate the depth of application-level monitoring. A healthy deployment should show services that match your known application inventory.

3.1 Service Count and Types

```
// Count services by service type
fetch dt.entity.service
| summarize service_count = count(), by:{serviceType}
| sort service_count desc
// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes SERVICE
// | summarize service_count = count(), by:{serviceType}
// | sort service_count desc
```

3.2 Process Group Inventory

```
// Count process groups by technology type
fetch dt.entity.process_group
| summarize pg_count = count(), by:{softwareTechnologies}
| sort pg_count desc
| limit 15
// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes PROCESS
// | summarize pg_count = count(), by:{softwareTechnologies}
// | sort pg_count desc
// | limit 15
```

4. Data Ingestion Rates

Understanding how much data flows into Dynatrace helps with capacity planning, cost management, and identifying gaps. A sudden drop in ingestion often signals an agent outage or configuration change.

4.1 Log Ingestion Volume Over Time

```
// Log ingestion trend over the last 24 hours by hour
fetch logs, from:-24h
| makeTimeseries log_count = count(), interval:1h
```

4.2 Log Ingestion by Source

```
// Top 10 log sources by volume in the last 24 hours
fetch logs, from:-24h
| summarize log_count = count(), by:{log.source}
| sort log_count desc
| limit 10
```

4.3 Span Ingestion Volume

```
// Span ingestion trend over the last 24 hours
fetch spans, from:-24h
| makeTimeseries span_count = count(), interval:1h
```

5. ActiveGate Health

ActiveGates serve as routing, monitoring extension, and API endpoints. Their health directly impacts data collection reliability.

5.1 ActiveGate Inventory

```
// List all ActiveGates with their properties
fetch dt.entity.environment_active_gate
| fieldsAdd entity.name
| summarize ag_count = count()
```

5.2 ActiveGate Metric Health

Self-monitoring metrics confirm ActiveGates are processing data. A flat or zero metric suggests the ActiveGate is unhealthy.

```
// ActiveGate connection count over the last 1 hour
timeseries connections = avg(dt.sfm.active_gate.connections), from:-1h, by:
{dt.entity.environment_active_gate}
```

6. License and Consumption Tracking

Tracking license consumption helps prevent unexpected overages and ensures you are getting value from your investment.

6.1 Host Unit Estimate

Host units are a primary consumption metric. Each host consumes units based on its memory size and monitoring mode.

```
// Estimate host unit consumption by monitoring mode
fetch dt.entity.host
| summarize host_count = count(), by:{monitoringMode}
| fieldsAdd estimated_hu = if(monitoringMode == "FULL_STACK", then:
host_count, else: host_count * 0)
```

6.2 Log Ingestion Volume for Cost Awareness

```
// Daily log record count over the last 7 days
fetch logs, from:-7d
| makeTimeseries daily_logs = count(), interval:1d
```

7. Building a Platform Health Scorecard

Combine the results from the queries above into a regular health scorecard. Review this scorecard weekly or monthly to track platform health trends.

Recommended Scorecard Metrics

Metric	Target	How to Measure
Host Coverage	> 95% of known hosts monitored	Compare entity count vs CMDB
Agent Version Currency	All agents within 1 major version	Version distribution query
Service Discovery	All known services auto-detected	Compare service count vs app inventory
Log Ingestion Stability	< 10% daily variance	7-day ingestion trend
Span Ingestion Active	> 0 spans per hour	Span count query
ActiveGate Health	All AGs reporting metrics	AG connection metrics
Dynatrace Intelligence Active	Problems detected in last 7 days	Problem count query
Alert Noise Ratio	< 20% duplicate/frequent	Noise ratio query from ADOPT-01

Scoring Guide

Score	Status	Action
8/8 metrics green	Healthy	Continue regular monitoring
6-7 green	Minor gaps	Address within current sprint
4-5 green	Significant gaps	Prioritize remediation
< 4 green	Critical	Escalate to platform team

8. Summary and Next Steps

Key Takeaways

- Platform health assessment should be a regular practice, not a one-time exercise
- OneAgent coverage and data ingestion stability are the two most critical health indicators
- A simple scorecard with 8-10 metrics provides actionable visibility into platform health
- Gaps discovered here directly inform your maturity roadmap from ADOPT-01

Next Steps

- Proceed to **ADOPT-03: Success Metrics** to define MTTR, MTTD, and other outcome-based metrics
- Schedule a recurring health scorecard review (weekly recommended for new deployments)
- Compare discovered entities against your CMDB to identify coverage gaps

---